



DIPARTIMENTO DI ELETTRONICA,
INFORMAZIONE E BIOINGEGNERIA

Politecnico di Milano

Machine Learning (Code: 097683)

June 20, 2026

Surname:

Name:

Student ID:

Row:

Column:

Time: 2 hours

Maximum Marks: 33

- The following exam is composed of **8 exercises** (one per page). The first page needs to be filled with your **name, surname and student ID**. The following pages should be used **only in the large squares** present on each page. Any solution provided either outside these spaces or **without a motivation** will not be considered for the final mark.
- During this exam, you are **not allowed to use electronic devices**, such as laptops, smartphones, smartwatches, tablets, and similar devices. Only non-programmable calculators are allowed. Any other device must be turned off and stored away for the entire duration of the exam.
- This is a closed-book/closed-notes exam. You are not allowed to bring with you any kind of note, book, written scheme, and/or similar. You are also not allowed to communicate with other students during the exam.
- Violation of the above-mentioned rules implies the immediate expulsion of the student from the exam room and a **REJECT** mark for the exam.
- Pencils are not allowed. It is your responsibility to provide a readable solution. We will not be held accountable for accidental partial or total cancellation of the exam. The exam can be written either in **English** or **Italian**.
- You are allowed to withdraw from the exam at any time without any penalty. You are not allowed to keep the text of the exam with you while leaving the room.
- **Three of the points will be given on the basis on how quick you are in solving the exam.** If you finish earlier than 30 min before the end of the exam, you will get 3 points; if you finish earlier than 20 min, you will get 2 points; and if you finish earlier than 10 min, you will get 1 point (the points cannot be accumulated).
- The box on Page 10 can only be used to complete the Exercises 7, and/or 8.

Ex. 1	Ex. 2	Ex. 3	Ex. 4	Ex. 5	Ex. 6	Ex. 7	Ex. 8	Time	Tot.
/ 7	/ 7	/ 2	/ 2	/ 2	/ 2	/ 4	/ 4	/ 3	/ 33

Exercise 1 (7 marks)

Consider *binary* logistic regression with weights $\mathbf{w}$, where the probability of the positive class for an input $\mathbf{x}$ is $p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$ and $\sigma(z) = \frac{1}{1+e^{-z}}$. Answer the following specific points.

1. Write the per-sample **cross-entropy (negative log-likelihood)** loss $\ell(\mathbf{w})$ for a labelled example $(\mathbf{x}, y)$ with $y \in \{0, 1\}$.
2. Derive the **gradient** $\nabla_{\mathbf{w}} \ell(\mathbf{w})$. (You may use the fact that $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.)
3. Explain in one or two sentences why the cross-entropy loss is preferred over the **squared error** $(y - \sigma(\mathbf{w}^\top \mathbf{x}))^2$ for training this model.

Exercise 2 (7 marks)

Consider temporal-difference *control* with a learning rate α and discount factor γ , in a transition $(s_t, a_t, r_{t+1}, s_{t+1})$ where the agent next selects action a_{t+1} . Answer the following specific points.

1. Write the **update rule of SARSA** and the **update rule of Q-learning** for the action-value estimate $Q(s_t, a_t)$, highlighting the only term in which they differ (the TD target).
2. Define what is meant by an **on-policy** and an **off-policy** algorithm; classify SARSA and Q-learning accordingly, and identify precisely *which element of the update rule* makes each algorithm on-policy or off-policy.
3. Give one concrete example in which SARSA and Q-learning may converge to **different** policies, and say what the difference is and why it arises.

Exercise 3 (2 marks)

Consider the following snippet of code, assuming that **X** contains numerical features (such as **is.holiday**, **temperature**, and **visibility_meters**) and **y** is a continuous target.

```
1 from sklearn.linear_model import Lasso
2 from sklearn.model_selection import train_test_split
3
4 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
5 model = Lasso(alpha=0.1)
6 model.fit(X_train, y_train)
7
8 importances = np.abs(model.coef_)
9 most_important = np.argmax(importances)
10 test_mse = mean_squared_error(y_test, model.predict(X_test))
```

1. Describe line by line what the snippet does and which learning task is solved.
2. What is the role of the hyperparameter **alpha**, and what impact can it have on the model?
3. There is a methodological flaw in this procedure. Identify it and propose a fix.

1. Line 4: the data is split 80/20 into train and test. Lines 5–6: a **Lasso** (linear regression with an ℓ_1 penalty) is trained on the training set with regularization strength **alpha**= 0.1. Lines 8–9: the absolute values of the learned coefficients are taken and the feature with the largest one is reported as the “most important”. Line 10: the model is evaluated with mean squared error on the test set. The task is **(regularized) linear regression**.
2. **alpha** is the regularization coefficient λ that weights the penalty term against the data-fit term. It controls the bias–variance trade-off: a larger **alpha** shrinks the coefficients more, giving a simpler, higher-bias / lower-variance model (and, if too large, underfitting); a smaller **alpha** approaches ordinary least squares, with lower bias but higher variance (risk of overfitting). (Specifically for Lasso, a large enough **alpha** also drives some coefficients exactly to zero, performing implicit feature selection.)
3. **The features are not normalized/standardized before applying Lasso.** The ℓ_1 penalty is *scale-dependent*: a feature measured on a large scale gets a correspondingly small coefficient (and is penalized less), while a small-scale feature gets a large coefficient. **Fix:** standardize the features (zero mean, unit variance) before fitting — fitting the **StandardScaler** on **X_train** only and applying it to **X_test** (ideally via a **Pipeline**) — so that the penalty acts comparably on all coefficients.

Exercise 4 (2 marks)

Tell whether the following statements about ensemble methods are true or false. Motivate your answers.

1. Bagging reduces variance mainly by averaging models trained on bootstrap samples, and is most effective with high-variance, low-bias base learners.
2. In boosting, the base learners are trained *independently* and in parallel.
3. In bagging, each base model is trained on a **bootstrap sample**, i.e. a dataset of the same size N obtained by sampling the training set *with replacement*.
4. Boosting can reduce the bias of weak learners, but if run for too many rounds it may overfit the training data.

1. **True.** Bagging averages predictions of models trained on bootstrap resamples, which reduces variance without increasing bias; it therefore benefits most high-variance, low-bias learners (e.g. fully grown decision trees), while it brings little benefit to stable, high-bias learners.
2. **False.** Boosting is **sequential**: each base learner is trained to correct the errors of the previous ones (e.g. by re-weighting misclassified samples in AdaBoost or fitting the residuals in gradient boosting), so they cannot be trained independently in parallel.
3. **True.** Bootstrapping draws N samples with replacement from the original N -sized training set, so each base model sees a slightly different dataset (some examples repeated, others absent — on average about 63% of the distinct examples appear). This diversity among the base models is exactly what makes their averaging reduce variance.
4. **True.** Boosting combines weak (high-bias) learners into a strong learner, primarily *reducing bias*; however, since it keeps fitting the training data, an excessive number of rounds can fit the noise and lead to overfitting (controlled via the number of rounds, learning rate, or regularization).

Exercise 5 (2 marks)

Tell whether the following statements about SVMs and kernels are true or false. Motivate your answers.

1. In a soft-margin SVM, removing a training point that is *not* a support vector leaves the decision boundary unchanged.
2. The function $k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + 1)^2$ is a valid kernel.
3. The function $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z} - 1$ is a valid kernel.
4. For the Gaussian (RBF) kernel $k(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$, decreasing the bandwidth parameter (i.e., making γ large) tends to increase the risk of overfitting.

1. **True.** The soft-margin solution depends only on the support vectors; non-support vectors have zero dual coefficient, so removing one does not change $\mathbf{w}$, b , or the boundary.
2. **True.** It is a polynomial kernel of degree 2: it can be written as an inner product $\phi(\mathbf{x})^\top \phi(\mathbf{z})$ in the feature space of all monomials up to degree 2, so the corresponding Gram matrix is positive semidefinite.
3. **False.** Note that a valid kernel *may* take negative values: what is required is not $k \geq 0$ but that, for *every* finite set of points $\{\mathbf{x}_i\}$, the Gram matrix $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ be symmetric **positive semidefinite** (Mercer's condition), equivalently that k corresponds to an inner product $\phi(\mathbf{x})^\top \phi(\mathbf{z})$ in some feature space. As a consequence, the diagonal must be non-negative, since $k(\mathbf{x}, \mathbf{x}) = \|\phi(\mathbf{x})\|^2 \geq 0$; but $k(\mathbf{x}, \mathbf{x}) = \mathbf{x}^\top \mathbf{x} - 1 = \|\mathbf{x}\|^2 - 1 < 0$ for any $\|\mathbf{x}\| < 1$. More directly, taking a single point with $\mathbf{x} = \mathbf{0}$ gives the 1×1 Gram matrix $[-1]$, which is not PSD. Hence k is not a valid kernel.
4. **True.** A large γ (small bandwidth) makes each point influence only a tiny neighbourhood, so the model can fit the training data very tightly (high variance), increasing the risk of overfitting.

Exercise 6 (2 marks)

You are a machine-learning specialist working for an online food-delivery company. For each of the following sub-problems, explain how you would frame it as a machine-learning problem, specifying the **input**, the **output / action space**, the **loss or reward**, and a **suitable method**.

1. From features of an order (distance, time of day, restaurant preparation time, weather, courier load), predict the **delivery time in minutes**.
2. For each new user, the app must choose which one of five promotional banners to display on the home screen; the only feedback available is whether the shown banner is clicked or not, and the click probability of each banner is unknown.

1. **Supervised regression.** Input: the order feature vector (distance, time of day, prep time, weather, courier load). Output: a continuous value, the delivery time in minutes. Loss: squared error (MSE) or MAE on a labelled dataset of past orders with known delivery times. Method: e.g. linear/Ridge regression, gradient-boosted trees, or a regression neural network; select hyperparameters by cross-validation.
2. **Multi-armed bandit** (with binary reward). Actions/arms: the 5 banners. Reward: 1 if the displayed banner is clicked, 0 otherwise. The click probabilities are unknown and only the chosen arm's reward is observed, so we must balance exploration and exploitation. Method: UCB1 or Thompson Sampling (Beta-Bernoulli) to minimise regret. (If user features are available, a contextual bandit / LinUCB is the natural extension.)

Exercise 7 (4 marks)

A learning algorithm selects a hypothesis from a *finite* hypothesis class H with $|H| = 150$. It is trained on $N = 400$ i.i.d. samples and the chosen hypothesis h has a **training (empirical) error** $\hat{L}(h) = 0.20$. We want a guarantee that holds with probability at least $1 - \delta$ with $\delta = 0.05$.

1. Write the agnostic generalization bound for a finite hypothesis class that upper-bounds the true error $L(h)$ in terms of $\hat{L}(h)$, $|H|$, N , δ .
2. Compute the resulting numerical upper bound on $L(h)$. (Use $\ln 150 \simeq 5$ and $\ln 20 \simeq 3$.)
3. How many samples N would be needed to guarantee that the gap between true and training error is at most 0.05 (same $|H|$ and δ)?
4. Qualitatively, how does the bound change if $|H|$ grows? Briefly justify.

1. With probability at least $1 - \delta$, for the selected h (union bound + Hoeffding over the finite class):

$$L(h) \leq \hat{L}(h) + \sqrt{\frac{\ln |H| + \ln \frac{1}{\delta}}{2N}}.$$

2. The numerator is $\ln |H| + \ln \frac{1}{\delta} = 5 + 3 = 8$, and $2N = 800$, so the gap term is

$$\sqrt{\frac{8}{800}} = \sqrt{0.01} \simeq 0.10.$$

Hence $L(h) \leq 0.20 + 0.10 \simeq \mathbf{0.30}$.

3. Impose $\sqrt{\frac{8}{2N}} \leq 0.05 \Rightarrow \frac{8}{2N} \leq 0.0025 \Rightarrow 2N \geq \frac{8}{0.0025} = 3200 \Rightarrow N \geq 1600$. So about $N \geq \mathbf{1600}$ samples.
4. A larger $|H|$ increases $\ln |H|$ and therefore *loosens* (raises) the bound: a richer class makes the uniform-convergence term larger, so more data is needed for the same guarantee. This is the capacity / overfitting penalty.

Exercise 8 (4 marks)

Consider a multi-armed bandit with binary rewards and two arms $\{a_1, a_2\}$, run for a horizon of $T = 8$ rounds. The table reports the reward R_t obtained at each round (only the reward of the played arm is revealed):

t	1	2	3	4	5	6	7	8
Reward from a_1	1	1		0		1		1
Reward from a_2			0		1		0	

1. Knowing that the expected rewards are $\mu_1 = 0.75$ and $\mu_2 = 0.25$, compute the expected regret over the horizon T .
2. Which arm would UCB1 play in the next round $t = 9$? Use $\sqrt{\frac{2\ln 8}{5}} \simeq 0.91$ and $\sqrt{\frac{2\ln 8}{3}} \simeq 1.18$. Motivate your answer.
3. Which arm is Thompson Sampling (uniform Beta prior at $t = 0$) more likely to play at $t = 9$? Give the two posterior distributions.

1. Arm a_1 is optimal. The suboptimal arm a_2 is played 3 times (at $t = 3, 5, 7$), so the expected regret is

$$n_2(\mu_1 - \mu_2) = 3 \times (0.75 - 0.25) = \mathbf{1.5}.$$

2. After 8 rounds: a_1 played $n_1 = 5$ times with rewards $\{1, 1, 0, 1, 1\}$ (empirical mean $\bar{x}_1 = 4/5 = 0.8$); a_2 played $n_2 = 3$ times with rewards $\{0, 1, 0\}$ (empirical mean $\bar{x}_2 = 1/3 \simeq 0.33$). The UCB1 indices are

$$U_1 = 0.8 + \sqrt{\frac{2\ln 8}{5}} \simeq 0.80 + 0.91 = 1.71,$$

$$U_2 = 0.33 + \sqrt{\frac{2\ln 8}{3}} \simeq 0.33 + 1.18 = 1.51.$$

Since $U_1 > U_2$, UCB1 plays **a₁**.

3. Updating the uniform Beta(1,1) prior with successes/failures: a_1 has 4 successes and 1 failure $\Rightarrow$ Beta(5,2); a_2 has 1 success and 2 failures $\Rightarrow$ Beta(2,3). The posterior of a_1 (mean $5/7 \simeq 0.71$) is concentrated on higher values than that of a_2 (mean $2/5 = 0.4$), so Thompson Sampling is more likely to play **a₁** (though, being randomised, not with certainty).

